SmartRoute AI: An Explainable Risk-Aware Deep Reinforcement Learning Framework for Dynamic Last-Mile Delivery Assignment

Sanjay T – 24619263
Sivanesh D – 24619252
YogaPriyan P – 24619260
VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES

MINI PROJECT REPORT

Submitted to the

SCHOOLOF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES

(VISTAS)

CHENNAI– 600117

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that the project report entitled "SmartRoute AI: An Explainable Risk-Aware Deep Reinforcement Learning Framework for Dynamic Last-Mile Delivery Assignment" is the work prepared by Sanjay T (24619263), Sivanesh D (24619252), and Yoga Priyan P (24619260) under the guidance of Ms. Usha, Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), VISTAS, Chennai.

This report documents the proposed system design, methodology, implementation plan, and evaluation strategy. The supplied source material identifies the framework as being at the design stage; therefore, quantitative performance improvements are not represented as completed experimental findings.

SIGNATURE OF THE HOD

Dr. K. Ulaga Priya Associate Professor & Head
Department of AI & ML
School of Engineering
VISTAS, Chennai

SIGNATURE OF THE PROJECT GUIDE

Ms. Usha
Department of AI & ML
School of Engineering
VISTAS, Chennai

Submitted for the Project Work and Viva-Voce examination held on ..... at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600 117.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We express our sincere gratitude to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, for providing the academic environment and facilities required to undertake this project. We thank the School of Engineering and the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning) for encouraging the application of artificial intelligence methods to practical engineering problems.

We are especially grateful to our project guide, Ms. Usha, for her guidance in structuring the problem, reviewing the proposed approach, and keeping the work focused on the operational needs of last-mile delivery. Her supervision helped us consider the importance of explainability, risk awareness, and systematic evaluation.

We also acknowledge the contributions of our faculty members, laboratory staff, classmates, and all those who offered feedback during project planning. Finally, we thank our families for their patience and support throughout this work.

Sanjay T – 24619263

Sivanesh D – 24619252

Yoga Priyan P – 24619260

TABLE OF CONTENT

ABSTRACT 6
LIST OF FIGURES 7
ABBREVIATIONS 8
CHAPTER 1 INTRODUCTION 9
1.1 Background and Project Overview 9
1.2 Problem Statement 9
1.3 Objectives 9
1.4 Scope and Expected Outcomes 9
1.5 Research Gap and Motivation 10
1.6 Operational Context of Dynamic Dispatch 10
1.7 Detailed Objectives and Success Indicators 10
1.8 Methodology Overview 11
1.9 Assumptions and Constraints 11
CHAPTER 2 LITERATURE REVIEW 12
2.1 Last-Mile Delivery Optimisation 12
2.2 Classical Assignment Methods 12
2.3 Deep Reinforcement Learning 12
2.4 Explainable and Safe AI 12
2.5 Summary and Research Gap 13
2.6 Assignment as a Markov Decision Process 13
2.7 Value-Based and Policy-Gradient Learning 13
2.8 Dynamic Routing and Dispatch Considerations 14
2.9 Interpretability in Operational Decision Support 14
2.10 Comparative Discussion of Related Approaches 14
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 15
3.1 Existing Approach and Limitations 15
3.2 Proposed System 15
3.3 System Architecture 15
3.4 Functional Requirements 16
3.5 Non-Functional Requirements 16
3.6 Hardware and Software Requirements 17
3.7 Stakeholders and Use Cases 17
3.8 Data Requirements and Data Model 17
3.9 Component-Level Design 18
3.10 Interface and Integration Design 18
3.11 Security, Privacy and Auditability 18
3.12 Design Decisions and Trade-offs 19
3.13 Failure Modes and Mitigation 19
3.14 Scalability Considerations 19
CHAPTER 4 METHODOLOGY AND IMPLEMENTATION 21
4.1 Development Methodology 21
4.2 State, Action and Environment Design 21

4.3 Risk-Aware Reward Function 21
4.4 DRL Agent and Baselines 22
4.5 Explainability Layer 22
4.6 Simulation and Data Flow 22
4.7 Implementation Modules 23
4.8 Scenario Generator Design 23
4.9 Feasibility Masking and Action Selection 24
4.10 Training Procedure 24
4.11 Hungarian Baseline Construction 25
4.12 Nearest-Partner Baseline Procedure 25
4.13 Explanation Generation Procedure 25
4.14 Logging, Configuration and Reproducibility 26
4.15 Implementation Plan 26
4.16 Worked Example of One Assignment Step 26
4.17 Pseudocode of the Decision Loop 27
CHAPTER 5 EVALUATION, TESTING AND DISCUSSION 28
5.1 Evaluation Metrics 28
5.2 Experimental Design 28
5.3 Test Plan 28
5.4 Expected Analysis and Limitations 29
5.5 Scenario Matrix 29
5.6 Statistical Treatment of Results 29
5.7 Planned Ablation Studies 29
5.8 Evaluation of Explanations 30
5.9 Threats to Validity 30
5.10 Acceptance Criteria for the Prototype 30
5.11 Reporting Format for Results 31
5.12 Resources and Runtime Considerations 31
CHAPTER 6 CONCLUSION AND FUTURE WORK 32
6.1 Conclusion 32
6.2 Future Enhancements 32
6.3 Recommendations for Final Validation 32
6.4 Ethical and Social Considerations 33
6.5 Summary of Contributions 33
6.6 Lessons for Practice 33
REFERENCES 35

ABSTRACT

Last-mile delivery is the final and operationally complex stage of a logistics chain. Orders arrive over time while road congestion, deadlines, partner availability, distance, and workload change continuously. SmartRoute AI is proposed as an explainable and risk-aware Deep Reinforcement Learning (DRL) framework for dynamic assignment of delivery orders to available partners. Its state representation combines order attributes, partner status, estimated travel time, distance, traffic conditions, and delivery priority.

A DRL agent learns an assignment policy through interaction with a software simulation environment. The reward design penalises delay, distance, operational risk, and workload imbalance while rewarding on-time delivery. The Hungarian Algorithm is retained as a classical baseline, and a nearest-partner rule provides a simple comparison. An explainability layer generates a concise reason for each assignment using travel time, risk, and workload.

The planned evaluation includes mean delivery time, on-time rate, assignment agreement with the Hungarian baseline on static snapshots, total distance, workload distribution, risk score, feasibility, and explanation coverage. Since the source paper identifies the project as being at the design stage, this report describes the proposed architecture and evaluation plan rather than claiming measured improvements.

Keywords: last-mile delivery, dynamic assignment, deep reinforcement learning, explainable AI, risk-aware optimisation, Hungarian Algorithm.

LIST OF FIGURES

Figure 1. High-level architecture of SmartRoute AI
Figure 2. Training loop of the DRL agent in the simulation environment

ABBREVIATIONS

AI	Artificial Intelligence
DRL	Deep Reinforcement Learning
DQN	Deep Q-Network
MDP	Markov Decision Process
PPO	Proximal Policy Optimisation
XAI	Explainable Artificial Intelligence SHapley
SHAP	Additive exPlanations
LIME	Local Interpretable Model-agnostic Explanations
RL	Reinforcement Learning

CHAPTER 1
INTRODUCTION

1.1 Background and Project Overview

Last-mile delivery refers to the movement of goods from a local distribution point to the final customer. Although it is the final stage of a shipment, it can be difficult to coordinate because many small deliveries must be completed across a road network with variable traffic and different deadlines. A dispatcher must select a suitable partner while considering current location, workload, availability, distance, and the likelihood of meeting the promised delivery time.

In a static setting, the dispatcher may create a cost matrix and select a low-cost assignment. Real delivery systems are less predictable: new orders arrive, partners complete earlier tasks, traffic changes, and urgent deliveries may need priority. A nearest-partner rule can also overload a subset of the fleet. SmartRoute AI treats assignment as a sequential decision process rather than one isolated calculation. The proposed system combines a DRL agent with a risk module, a Hungarian Algorithm baseline, and an explainability layer. The agent observes the current environment and selects a feasible order–partner assignment. A reward function encodes operational trade-offs, while the explanation layer describes the main factors behind the selected partner. The initial study is designed for software simulation and does not require physical delivery hardware.

1.2 Problem Statement

Delivery assignment must balance delivery time, travel distance, partner availability, deadlines, congestion, and workload. Static optimisation can produce a good assignment for a fixed snapshot, but its result may lose quality when conditions change. Simple rules are fast but may ignore future workload and combined risk. A learned model can adapt to experience, yet a black-box decision is difficult to audit without a useful explanation. The problem is to design a dynamic assignment framework that considers risk, compares with established baselines, and explains each decision.

1.3 Objectives

The project objectives are to train a DRL policy for dynamic assignment; represent order, partner, traffic, and deadline information; include congestion, delay risk, excess distance, and workload in a risk score; design a reward that balances service quality and operational cost; compare against the Hungarian Algorithm and nearest-partner rule; and produce human-readable explanations.

1.4 Scope and Expected Outcomes

The scope covers a simulator, state representation, risk-aware reward calculation, candidate DRL policy, baseline methods, explanation generation, and repeatable evaluation. It does not yet constitute a complete production routing platform. Expected outcomes include a reproducible software framework and evidence about trade-offs under light, moderate, and heavy demand. A learned policy should not be assumed to outperform every baseline on every metric.

1.5 Research Gap and Motivation

Prior work covers vehicle-routing optimisation, classical assignment, reinforcement learning for routing and dispatch, safe reinforcement learning, and explainable reinforcement learning. These directions are often evaluated separately. SmartRoute AI integrates dynamic assignment, explicit risk, workload balance, baseline comparison, and per-decision explanation. Its contribution remains a proposed integrated design until controlled experiments validate its behaviour.

- Dynamic adaptation to order arrivals and traffic changes.
- Risk-aware scoring alongside time and distance.
- Workload balancing to avoid excessive concentration of assignments.
- A recognised optimisation baseline and a simple heuristic baseline.
- Decision explanations suitable for dispatcher review.

1.6 Operational Context of Dynamic Dispatch

Delivery platforms receive requests continuously rather than in a single batch. Each request carries a pickup or drop location, a promised delivery window, and in many cases a priority level such as standard, express, or time-critical. At any moment a dispatcher must decide which of the currently available partners should accept an order, and this decision must be made quickly because customers and merchants expect prompt confirmation. The quality of a decision depends on information that changes between one decision and the next: the position of every partner, the number of orders each partner is already carrying, and the traffic conditions on candidate routes.

Because decisions are taken one after another, an early choice influences the options that remain. Assigning a nearby partner to an order may remove that partner from consideration for a more urgent request that arrives a minute later. Likewise, repeatedly assigning work to the same partner may produce fatigue, late deliveries, or unfair earnings. These effects are not visible in a one-shot cost matrix, which is the main reason this project formulates assignment as a sequential decision problem and not as a collection of independent calculations.

Operational teams also require accountability. When a delivery is late, a supervisor may need to know why a particular partner was chosen and whether a better alternative existed. A system that cannot give a short and truthful reason for its decisions is difficult to adopt even if its aggregate performance is good. SmartRoute AI therefore treats explanation as part of the decision pipeline and not as an afterthought.

1.7 Detailed Objectives and Success Indicators

The objectives listed in Section 1.3 are refined below into statements that can be verified once the simulator and agent have been implemented. Each indicator describes what must be demonstrated, not a result that has already been obtained.

- Policy learning: a DQN or PPO agent can be trained in the simulator without invalid actions, and its training curves are recorded for several random seeds.
- State representation: every observation contains order, partner, traffic, distance, and deadline features in a documented and scaled form.

- Risk modelling: the risk score is computed from congestion, delay likelihood, excess distance, and workload, and each component is bounded on a documented scale.
- Reward design: the reward exposes separate delay, distance, risk, workload, and on-time terms so that each can be logged and analysed.
- Baseline comparison: the Hungarian Algorithm and the nearest-partner rule are run on the same scenarios, constraints, and traffic realisations as the learned policy.
- Explainability: every assignment is accompanied by a short explanation derived from measured feature differences between the selected partner and feasible alternatives.
- Reproducibility: a result can be regenerated from a stored configuration, random seed, and model version.

1.8 Methodology Overview

The work is organised as a design-first study. The first stage defines the problem, the entities, and the constraints. The second stage specifies the state, action, reward, and risk formulations. The third stage describes the simulator, the learning agent, the baselines, and the explanation procedure. The final stage defines how the methods will be compared, what statistical treatment is appropriate, and which limitations must be reported. This ordering allows each design decision to be reviewed before expensive training runs are started.

Throughout the report, statements about expected behaviour are written as expectations or hypotheses. No measured improvement is claimed, because the framework is presented at the design stage and the controlled experiments described in Chapter 5 remain to be carried out.

1.9 Assumptions and Constraints

The following assumptions simplify the first version of the framework. They are stated explicitly so that readers can judge how far the proposed design can be generalised and which assumptions must be relaxed in later work.

- Each order is assigned to exactly one partner, and a partner may hold a bounded number of active orders.
- Travel-time estimates are produced by the simulator from distance and a time-varying traffic multiplier, not by a live map service.
- Partners are cooperative and always accept an assignment that is offered to them.
- Order arrivals follow configurable stochastic processes, and the scenario generator is controlled by a random seed.
- The study is software-only; no physical vehicles, handsets, or customer data are required.

Practical constraints include limited computing resources, the time available for a mini project, and the absence of a real delivery dataset. These constraints favour a modest network size, a small to medium fleet in the simulator, and a clear separation between what is designed, what is implemented, and what has been measured.

CHAPTER 2
LITERATURE REVIEW

2.1 Last-Mile Delivery Optimisation

Dantzig and Ramser introduced the truck dispatching problem, which became a foundation for vehicle-routing research [15]. Boysen, Fedtke, and Schwerdfeger surveyed last-mile delivery concepts and described the operational pressure created by customer time expectations and diverse delivery models [1]. These works establish the importance of efficient planning, but they do not by themselves provide an adaptive policy for every newly arriving order.

A delivery decision depends on the state at the moment of assignment. A nearby partner may already carry several orders, while a more distant partner may have a lower workload or a more reliable path under current congestion. Distance and time matter, but so do deadlines, availability, risk, and the effect of the decision on future orders.

2.2 Classical Assignment Methods

Kuhn's Hungarian method solves a one-to-one assignment problem optimally for a given cost matrix [2]. It is a useful baseline for SmartRoute AI because it provides a recognised reference for a static snapshot. The matrix can combine estimated travel time and risk. The method is optimal for the supplied matrix and constraints; a new order, traffic update, or workload change may alter that matrix and require recalculation.

2.3 Deep Reinforcement Learning

Reinforcement learning models an agent that observes a state, chooses an action, receives a reward, and transitions to a new state. Sutton and Barto provide the standard theoretical treatment [16]. Deep Q-networks learn control policies from experience [3], while Proximal Policy Optimisation is a policy-gradient approach designed for stable updates [4]. These are candidate methods for the assignment agent. Nazari et al. studied reinforcement learning for vehicle-routing problems [5], and Kool et al. investigated attention-based route construction [6]. Xu et al. explored learning and planning for large-scale ride-hailing dispatch [7], while Lin et al.

studied multi-agent reinforcement learning for fleet management [8]. These works support the potential of learning-based logistics, but their tasks and constraints differ from continuous last- mile assignment.

2.4 Explainable and Safe AI

Puiutta and Veith surveyed explainable reinforcement learning [11], and Heuillet et al. reviewed explainability in deep reinforcement learning [12]. SHAP explains predictions through feature contributions [9], while LIME explains an individual prediction using a locally interpretable approximation [10]. García and Fernández surveyed safe reinforcement learning [13], and Achiam et al. proposed constrained policy optimisation [14]. In SmartRoute AI, risk is represented through congestion, delay likelihood, excess distance, and current workload. This score is a design component, not a guarantee of safety.

2.5 Summary and Research Gap

The literature falls into classical assignment and vehicle-routing optimisation [1], [2], [15], learning-based routing and dispatch [3]–[8], [16], and explainable or risk-constrained learning [9]–[14]. SmartRoute AI combines these ideas within one framework. The main research gap is the integration of dynamic assignment, operational risk, workload balance, a classical baseline, and human-readable explanations. Measured superiority remains an open question until experiments are completed.

2.6 Assignment as a Markov Decision Process

A Markov Decision Process (MDP) is defined by a set of states, a set of actions, a transition function, a reward function, and a discount factor. In the delivery setting, a state contains the pending orders, the partner positions and loads, and the current traffic profile. An action assigns a pending order to one eligible partner. The transition function describes how the simulator advances time, updates workload, introduces new orders, and changes travel times. The reward function converts the immediate consequences of an assignment into a scalar signal. Sutton and Barto describe this framework and the associated notions of return and value functions in detail [16].

The discount factor controls how strongly the agent values future consequences. A value close to one encourages the agent to consider the effect of an assignment on later orders, which is important when partner availability is limited. A small value makes the policy short-sighted and closer to a greedy rule. The choice of discount factor is therefore a design parameter that should be examined during validation instead of being fixed arbitrarily.

The Markov property requires the state to contain all information needed to predict the future. If the state omits partner workload or traffic level, the agent cannot learn their influence and the problem becomes partially observable. This observation motivates the inclusion of workload, deadline, and traffic features in the state vector and supports the requirement that the state builder be tested independently.

2.7 Value-Based and Policy-Gradient Learning

Deep Q-Networks approximate the action-value function with a neural network and learn from stored transitions using a replay buffer and a periodically updated target network [3]. They are well suited to discrete action spaces, such as choosing one partner from a list. Their main weaknesses are sensitivity to hyperparameters, overestimation of action values, and difficulty when the number of actions changes from step to step.

Proximal Policy Optimisation belongs to the policy-gradient family. It optimises a clipped surrogate objective that limits the size of each policy update, which usually improves training stability [4]. PPO can represent a stochastic policy and can be combined with action masking so that infeasible partners receive zero probability. It typically needs more environment interaction than a value-based method but is often easier to tune.

For SmartRoute AI both algorithms are retained as candidates. The final selection should be based on observed stability across seeds, compatibility with variable numbers of eligible partners,

training time, and performance on held-out scenarios. Selecting an algorithm before such evidence is available would be premature.

2.8 Dynamic Routing and Dispatch Considerations

Research on the vehicle-routing problem began with the truck dispatching formulation of Dantzig and Ramser [15] and has since expanded to include time windows, heterogeneous fleets, and stochastic demand. Learning-based approaches such as the pointer-network method of Nazari et al. [5] and the attention model of Kool et al. [6] construct routes sequentially and can generalise across instances drawn from a distribution. Large-scale dispatch systems, such as those studied by Xu et al. [7] and Lin et al. [8], combine learned value estimates with planning or coordination to manage thousands of vehicles.

These studies are valuable precedents, but they differ from the present problem in important ways. Ride-hailing dispatch typically optimises passenger waiting time and driver idle time, whereas last-mile delivery adds parcel deadlines, multi-order workloads, and priority classes. Routing methods optimise the sequence of stops for a given vehicle, while SmartRoute AI concentrates on the earlier question of which partner should receive each order. The two problems are complementary, and joint treatment is identified as future work.

2.9 Interpretability in Operational Decision Support

An explanation is useful only if it is faithful to the decision and understandable to the person reading it. Surveys of explainable reinforcement learning distinguish between methods that explain individual decisions, methods that summarise policy behaviour, and methods that make the learning process itself transparent [11], [12]. Feature-attribution techniques such as SHAP [9] and LIME [10] fit the first category, and they have been widely applied to supervised models.

Applying such techniques to sequential decision making raises additional issues. The same feature may matter differently at different times, and an attribution computed around one state may not describe the policy elsewhere. For this reason the proposed explanation layer begins with direct comparison of travel time, risk, and workload between the selected partner and the alternatives, which is simple and easy to audit. Attribution methods are considered as optional extensions whose stability must be checked.

2.10 Comparative Discussion of Related Approaches

Classical assignment methods guarantee optimality for a fixed cost matrix but do not model the future. Heuristic rules are fast and transparent but ignore interactions between decisions. Learning-based dispatch can capture temporal effects, yet it requires a simulator, careful reward engineering, and a way to build trust in its choices. Safe reinforcement learning introduces constraints or risk measures [13], [14], but is frequently studied in control tasks instead of delivery operations.

No single approach covers every requirement. The proposed framework keeps the optimisation baseline as a reference, uses a learned policy to handle sequential effects, expresses risk as part of the reward and the state, and generates explanations for each decision. The comparison in later chapters is intended to reveal where each method is strong and where it is weak, not to assume the superiority of the learned policy.

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Existing Approach and Limitations

Conventional delivery assignment may rely on dispatcher judgement, nearest-partner selection, or optimisation over a fixed snapshot. Manual judgement can incorporate local knowledge but may be inconsistent under high demand. Nearest-partner assignment is inexpensive yet may overlook deadlines, existing workload, and traffic risk. Snapshot optimisation minimises the supplied matrix but does not automatically learn how current decisions affect later states.

- Assignments may become outdated when traffic, order volume, or availability changes.
- Distance-only rules may overlook deadlines, congestion, and workload.
- Repeated selection of the closest partner can create uneven utilisation.
- Decisions may be difficult to audit without recorded explanations.
- Results may be unreliable if scenarios and random seeds are not controlled.

3.2 Proposed System

SmartRoute AI accepts delivery requests, partner status, traffic estimates, and distance or travel-time information. A state builder combines the inputs. A risk module calculates a normalised score for candidate assignments. The DRL agent selects among feasible actions, while the Hungarian Algorithm and nearest-partner rule provide comparison points. The output layer records the assignment, explanation, and evaluation metrics.

3.3 System Architecture

The input layer provides delivery requests, partner state, and traffic/distance estimates. The decision layer builds the state, calculates risk, and runs the DRL policy or a baseline. The output layer records the assignment and produces explanations and metrics. Delivery Requests location, deadline, priority Partner Status

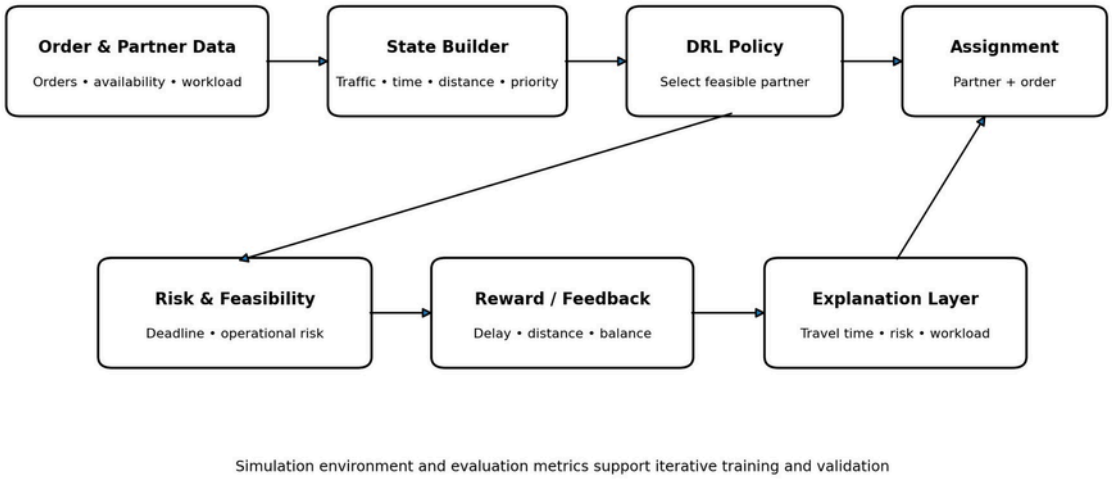


Figure 1. High-level architecture of SmartRoute AI

position, load, availability Traffic &Distancetravel-timeestimates State Builder combine
order, partner and traffic state Risk Module congestion, delay, distance, workload Decision Methods DRL agent + Hungarian baseline Assignment Decision selected feasible partner Explainability & Metrics reason, risk, time, workload

Figure 1. High-level architecture of SmartRoute AI. This is the proposed software design; integration and runtime performance

require implementation and testing.

3.4 Functional Requirements

The system must generate orders with location, deadline, priority, and arrival time; maintain partner position, availability, and workload; estimate travel time and distance; build state observations; calculate risk scores; select feasible assignments; run baseline methods on equivalent scenarios; record outcomes and reward components; generate explanations; and export summaries across scenario settings and random seeds.

3.5 Non-Functional Requirements

Reproducibility requires storing random seeds, configuration, model version, and evaluation settings. Reliability requires explicit handling of infeasible assignments and missing features. Interpretability requires explanations to reflect the actual decision factors. Scalability requires practical state construction and candidate scoring as the fleet grows. Maintainability requires clear interfaces between simulator, policy, baseline, risk, explanation, and reporting modules. Data integrity requires a clear distinction between simulated and real observations.

3.6 Hardware and Software Requirements

Adevelopment computer capable of running Python, scientific libraries, and model training is sufficient for the initial prototype. A GPU may shorten training for larger networks but is not mandatory. The planned stack includes Python, PyTorch, a Gym-compatible environment, and a reinforcement-learning library such as Stable-Baselines3. NumPy and pandas can support data handling, and plotting tools can be used after experiments are completed.

Component Purpose Python Simulator, reward, policy, and evaluation PyTorch Neural-network implementation Gym-compatible interface Standard environment reset and step behaviour Stable-Baselines3 (candidate) DRL algorithm implementations NumPy / pandas Scenario data and analysis Version control Track code and experiment configurations

3.7 Stakeholders and Use Cases

Four groups of stakeholders are considered. Dispatchers use the system to assign and monitor orders and need clear reasons for each recommendation. Delivery partners are affected by the fairness and feasibility of their assignments. Customers benefit from reliable delivery within the promised time. Managers and analysts use the evaluation reports to compare methods and decide whether a policy is suitable for wider use.

- Assign a new order: the dispatcher submits an order and the system returns a recommended partner together with an explanation.
- Review an assignment: the dispatcher inspects the travel time, risk score, and workload of the selected partner and the closest alternatives.
- Compare methods: an analyst runs the learned policy, the Hungarian baseline, and the nearest-partner rule on the same scenario and reads the metric summary.
- Audit a decision: a manager retrieves the logged state, action, reward components, model version, and explanation for a past assignment.

3.8 Data Requirements and Data Model

The first version of the framework works with synthetic data produced by the scenario generator. The data model is nevertheless designed so that real records could replace the synthetic ones later. Four principal entities are required.

- Order: identifier, pickup and drop-off coordinates, arrival time, deadline, priority, status, and assigned partner.
- Partner: identifier, current coordinates, availability flag, number of active orders, capacity limit, and cumulative workload.
- Traffic profile: time-dependent multipliers that scale base travel times for zones or for the whole network.
- Decision record: time, state summary, feasible candidates, selected partner, reward components, explanation text, method name, configuration identifier, and random seed.

Distances can be computed from coordinates, with a road factor applied to approximate the extra length of road paths compared with straight-line distance. The appendix illustrates this approximation. Synthetic traffic is generated from a small number of profiles, for example off-

peak, morning peak, and evening peak, so that the effect of congestion can be studied in a controlled way.

3.9 Component-Level Design

3.9.1 State Builder

The state builder collects order, partner, and traffic information and converts it into a numerical observation. It scales distances, times, and workloads to comparable ranges and pads or truncates lists when the number of orders or partners differs between steps. It also produces a feasibility mask that marks which order–partner pairs are allowed.

3.9.2 Risk Module

The risk module receives a candidate assignment and returns a normalised score between zero and one. The score aggregates congestion on the expected route, the estimated probability of missing the deadline, excess distance beyond a reference path, and the current workload of the partner. The aggregation weights are configuration parameters and are recorded with every experiment.

3.9.3 Decision Methods

The decision layer contains three interchangeable methods that share the same interface: the DRL policy, the Hungarian Algorithm, and the nearest-partner rule. A common interface ensures that each method receives identical state information and that any difference in outcome is due to the decision logic rather than to the input.

3.9.4 Explanation and Metrics Modules

The explanation module compares the selected partner with the feasible alternatives and produces a sentence that states the dominant reasons. The metrics module records delivery time, on-time status, distance, workload, risk, feasibility, and explanation coverage for every completed order and aggregates them by method and scenario.

3.10 Interface and Integration Design

The simulator is exposed to the learning library through the reset and step operations of a Gym-compatible environment [17]. The reset operation initialises a scenario from a configuration and a seed and returns the first observation. The step operation applies an assignment, advances the simulation, and returns the next observation, the reward, a termination flag, and an information dictionary containing the individual reward components.

A thin service layer can wrap the same logic so that a dispatcher interface could request a recommendation for a new order and receive the selected partner, the risk score, and the explanation. The documented prototype uses a web backend for this purpose, but the core framework does not depend on a particular web technology and can be exercised entirely from scripts during experiments.

3.11 Security, Privacy and Auditability

Even a simulation-based project should anticipate the requirements of an operational system. Order and partner records may contain personal information, so any later use of real data must follow the applicable privacy rules, store only the fields that are needed, and restrict access by role. Authentication and session handling must be tested, including failed logins, expired sessions, invalid tokens, and rate limiting.

Auditability is supported by immutable decision records. Each record links the decision to the model version and configuration that produced it, which allows a reviewer to reproduce the situation and to determine whether an unusual assignment was caused by the input, the reward weights, or the learned policy.

3.12 Design Decisions and Trade-offs

Several design decisions involve trade-offs that are recorded here for transparency. First, a single scalar reward is easy to optimise but hides the relative importance of its components; the framework therefore also logs the components separately. Second, a rich state improves the information available to the agent but enlarges the network and the training cost; scaling and a bounded candidate list limit this growth. Third, a simulator is repeatable and safe, but it cannot reproduce all real-world behaviour; results must therefore be described as simulation findings.

Finally, the explanation layer favours direct feature comparison over complex attribution because simple explanations are easier to verify. This choice may omit subtle interactions, and the trade-off is accepted for the initial version.

3.13 Failure Modes and Mitigation

A dispatch framework should be designed with its failure modes in mind. Each mitigation listed below can be tied to a planned test.

- Missing or invalid input: an order without a valid deadline or a partner without a position is rejected at the state builder and logged, so that incomplete records never reach the policy.
- No feasible partner: the environment records a deferral, keeps the order queued, and applies the configured waiting penalty instead of forcing an invalid assignment.
- Extreme traffic values: travel-time multipliers are clipped to a documented range, which prevents a single abnormal estimate from dominating the cost or the reward.
- Policy failure: if the learned policy returns an invalid action or violates a constraint, the nearest-partner rule is used as a fallback and the event is recorded for analysis.
- Explanation mismatch: if an explanation cannot be generated from the logged features, the assignment is flagged instead of being given a generic sentence.

Each of these cases corresponds to at least one test in the plan of Section 5.3, which allows the behaviour of the framework to be checked against the intended handling.

3.14 Scalability Considerations

The number of order–partner pairs grows with both the number of pending orders and the size of the fleet. A direct representation of all pairs may become expensive when the fleet is large.

Three measures can limit the cost. First, candidate pruning keeps only the partners within a reasonable travel time of the order. Second, the action space can be defined per order, so the agent chooses a partner for one order at a time. Third, the cost matrix of the Hungarian Algorithm can be restricted to the pruned candidate set, which keeps the cubic complexity of the method manageable.

The effect of pruning on solution quality is an empirical question. A very tight pruning radius may remove the partner that would have been the best choice under heavy congestion, whereas a loose radius increases computation. The pruning parameter is therefore included in the configuration and examined in the sensitivity analysis.

CHAPTER 4
METHODOLOGY AND IMPLEMENTATION

4.1 Development Methodology

The project follows an iterativeexperimental methodology. First, the assignment problem and constraints are defined. Next, a simulator generates orders, partner states, and traffic changes. State and reward functions are tested independently with controlled examples. A simple baseline is used to validate environment behaviour before training the DRL policy. The learned policy is evaluated on held-out scenarios and compared with baselines using common metrics.

Training and evaluation must remain separate. During training, the agent interacts with the environment and updates its policy from reward feedback. During evaluation, model parameters are frozen and scenarios are not used for further tuning. Multiple random seeds should be used, with means and variability reported. DQN and PPO are candidate approaches; the final choice should consider action-space compatibility, stability, training cost, and performance across scenarios.

4.2 State, Action and Environment Design

At time t, state sI summarises pending order locations, deadlines and priorities; partner positions, availability and workload; and traffic, distance, and travel-time estimates. Numerical features should be scaled when their magnitudes differ substantially. The action aI assigns an order to an available partner. Only feasible pairs should be selectable. If no partner is eligible, the simulator should record a defined no-assignment or deferral event.

After an action, the simulator updates partner workload, task progress, order status, and time. It introduces new orders according to the scenario generator and changes traffic multipliers according to the selected traffic profile. Completed or overdue tasks update their records and contribute to the next reward. These transitions make assignment sequential because each decision changes the state for later orders.

4.3 Risk-Aware Reward Function

The proposed reward combines penalties for delivery delay, travel distance, operational risk, and workload imbalance with a bonus for on-time delivery: $rI = -(\alpha \cdot DI + \beta \cdot LI + \gamma \cdot KI + \delta \cdot BI) + R_on\text{-}time$ Here, DI is delay beyond the deadline, LI is travel distance, KI is the combined risk score, BI measures workload imbalance, and α, β, γ, and δ are tunable weights. R_on-time is the configured bonus for meeting the deadline. The weights must be selected through documented validation rather than adjusted only to make one model appear better.

4.3.1 Risk Score Components

Risk K is a normalised combination of congestion along the expected route, the probability of delay, excess travel distance, and current partner workload. Components should be mapped to comparable scales before aggregation; otherwise, a factor with larger units can dominate for mathematical rather than operational reasons.

4.3.2 Weight Selection and Sensitivity

Weights should be tuned on training or validation scenarios and fixed before final evaluation. Sensitivity analysis should vary weights and record changes in delivery time, distance, workload, and risk. A single weighted reward can conceal trade-offs, so individual metrics must be reported alongside cumulative reward.

4.3.3 Reward Design Risks

An incomplete reward can encourage unintended behaviour. A distance penalty that is too large may favour short trips despite urgent deadlines; a workload penalty that is too strong may send an order to a distant partner. Tests should include high-priority orders, heavy congestion, few available partners, and simultaneous arrivals. Risk reduction must be measured as an outcome rather than inferred from the existence of a risk term.

4.4 DRL Agent and Baselines

A neural network scores feasible assignment actions from the current state. DQN estimates action values, whereas PPO learns a policy through policy-gradient updates. The Hungarian Algorithm is applied to a cost matrix built from travel time and risk for a static snapshot. A nearest-partner rule selects the eligible partner with the smallest configured distance or travel-time estimate. All methods must use equivalent feasibility constraints and scenario information.

The Hungarian baseline is a reference for the matrix it receives, not proof that every DRL decision should match it. Agreement with the Hungarian assignment is a comparison metric, not absolute optimality in a dynamic environment. The nearest-partner rule establishes whether added model complexity provides practical value.

4.5 Explainability Layer

For each assignment, the explanation layer compares the selected partner with feasible alternatives using estimated travel time, risk score, and workload. It can state that the partner was selected because travel time was lower while risk remained acceptable, or because the choice avoided adding work to an already busy partner. The wording must reflect measured feature differences and should not claim that a partner is safest or fastest unless the comparison supports that statement.

Because the reward contains named components, the system can report how the selected action scores on delay, distance, risk, and workload. SHAP or LIME may be applied for deeper feature attribution [9], [10]. Attribution methods are optional additions and should be checked for stability and fidelity. They do not replace basic testing of whether the explanation accurately reflects the model decision.

4.6 Simulation and Data Flow

The simulator generates orders over time, places delivery partners on a city representation,
changes travel times using traffic multipliers, and records deadlines and completions. A Gym- compatible interface is planned so the environment can expose reset and step operations to the DRL library. Python and PyTorch are proposed for implementation, with Stable-Baselines3 as a

candidate library [17]–[19]. Simulated Environment orders, partners, traffic Observe State s(t) current operating conditions DRL Policy / Q-Network score feasible actions

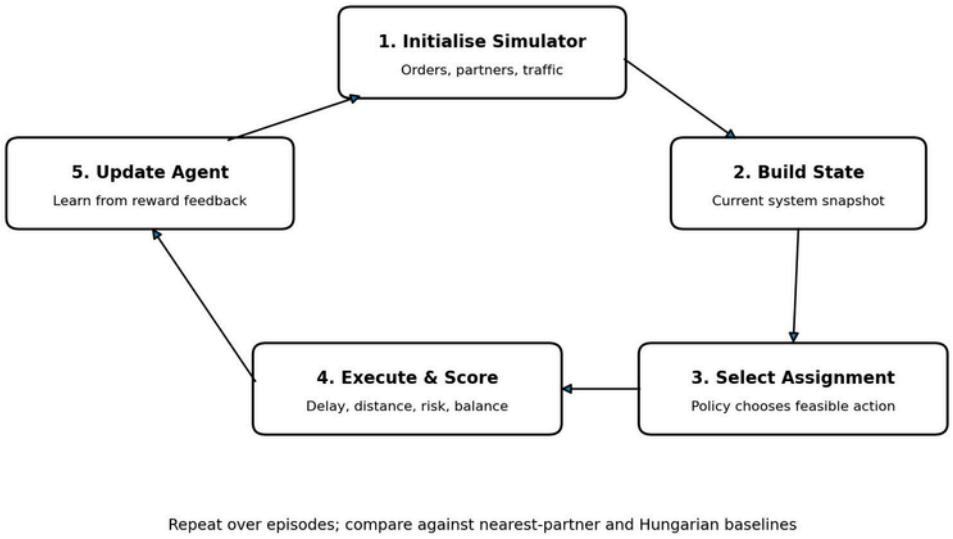


Figure 2. Training loop of the DRL agent in the simulation environment

Choose Action a(t) assign partner to order Reward r(t) delay, distance, risk, balance policy update

Figure 2. Training loop of the DRL agent in the simulation environment. Reward feedback updates the policy during training.

4.7 Implementation Modules

The order module stores order ID, location, arrival time, deadline, priority, and status. The partner module stores partner ID, location, availability, and workload. The traffic module defines time-varying multipliers and travel-time estimates. The risk module normalises risk factors. The assignment module executes the DRL or baseline policy. The explanation module generates a reason from relevant feature comparisons. The logging module records configuration, random seed, action, reward components, outcomes, and explanation.

Each simulator step should update state consistently. A partner must not remain available after being assigned if the simulated rules say the partner is occupied. Paired comparisons should use the same traffic realisation for all methods. Evaluation logs should be retained so unexpected results can be investigated instead of being hidden by aggregate averages.

4.8 Scenario Generator Design

The scenario generator produces reproducible sequences of orders, partner states, and traffic conditions. A scenario is defined by the length of the simulated period, the number of partners, the mean order arrival rate, the spatial distribution of pickup and drop-off points, the distribution of deadline slack, the proportion of priority orders, and the traffic profile. All random draws use a seeded generator so that the same scenario can be repeated for every method.

Three demand levels are planned. In the light setting the number of partners exceeds the number of simultaneously pending orders, so most orders have several feasible candidates. In the moderate setting the fleet is used at a substantial fraction of its capacity. In the heavy setting orders frequently compete for partners, deadlines are tight, and feasibility constraints are often binding. Each demand level can be combined with normal or congested traffic.

Spatial layouts may be uniform, clustered around a few demand centres, or mixed. Clustered layouts are useful for testing whether the policy balances workload when many orders originate from the same area. Deadline slack is drawn from a distribution whose lower tail produces occasional urgent orders, allowing the effect of priority handling to be examined.

4.9 Feasibility Masking and Action Selection

A pair of order and partner is feasible when the partner is available, has remaining capacity, and can reach the order within an acceptable travel time under the current traffic profile. The feasibility mask is a binary vector over the candidate actions. During training and evaluation, actions with a mask value of zero are removed from consideration so that the agent cannot select an invalid partner.

For value-based learning, the Q-values of masked actions are replaced by a very low number before the maximum is taken. For policy-gradient learning, the logits of masked actions are set to a very low value before the softmax, which yields zero probability for them. If every candidate is masked, the environment records a deferral event and keeps the order in the queue, optionally with a penalty that reflects the waiting cost.

Using a mask rather than penalising invalid actions after the fact shortens training and removes a source of unintended behaviour, because the agent does not need to learn constraints that the simulator can enforce directly.

4.10 Training Procedure

Training is carried out in episodes. At the start of an episode the environment is reset with a scenario configuration and a seed. The agent observes the state, selects a feasible action, and receives the reward and next state. Transitions are stored in a buffer for DQN or collected in rollouts for PPO. The network parameters are updated at regular intervals until the planned number of environment steps has been completed.

4.10.1 Hyperparameters

The main hyperparameters are the learning rate, discount factor, batch size, network width and depth, exploration schedule for DQN, clipping range and number of epochs per update for PPO, and the reward weights. Initial values should be taken from common practice and adjusted using validation scenarios only. Every value is stored with the experiment so that results can be reproduced.

4.10.2 Validation and Model Selection

Validation scenarios are generated with seeds that are different from those used for training and for the final test. The checkpoint with the best validation performance on a pre-defined criterion is retained, and the criterion must be chosen before training begins. If several criteria are

used, they should be combined according to a documented rule so that the selection does not depend on the later test results.

4.10.3 Training Diagnostics

Useful diagnostics include the episode return, the individual reward components, the proportion of on-time deliveries, the number of deferrals, the entropy of the policy for PPO, and the temporal-difference error for DQN. A policy that reaches a high return by neglecting one component, for example by ignoring workload balance, should be detected through these indicators and not accepted solely on the basis of the cumulative reward.

4.11 Hungarian Baseline Construction

For the static baseline, the pending orders and the available partners at a decision time are arranged in a cost matrix. Each entry combines the estimated travel time and the risk score for the corresponding pair, using the same weights as the reward where appropriate. Infeasible pairs receive a very large cost so that they are not chosen when a feasible alternative exists. If there are more orders than partners, or the reverse, the matrix is padded with dummy rows or columns to make it square [2].

The Hungarian Algorithm then returns the assignment with the minimum total cost for this matrix. Because new orders arrive later, the algorithm is invoked again at each decision time with the updated state. This rolling use is a reasonable baseline for dynamic operation but is not equivalent to solving the entire future problem, and the report should describe it as such.

4.12 Nearest-Partner Baseline Procedure

The nearest-partner rule examines the feasible partners for an order and selects the one with the smallest distance or estimated travel time. Ties are broken by a fixed rule, for example by the lower current workload and then by identifier, so that results are repeatable. The rule requires no training and negligible computation, which makes it a useful lower bound on the complexity that a learned method must justify.

4.13 Explanation Generation Procedure

The explanation is produced after the decision has been made and uses only quantities that were available when the decision was taken. The procedure has four steps.

- Collect the feasible candidates and their travel time, risk score, and workload.
- Compute the difference between the selected candidate and the best alternative for each of the three factors.
- Choose the factors whose differences are largest or most favourable, subject to configured thresholds that prevent trivial differences from being reported.
- Fill a sentence template with the chosen factors, for example by stating that the selected partner has the lower estimated travel time while keeping risk below the configured threshold.

Templates are used because they guarantee that the wording corresponds to the data. The explanation must not state that a partner is the safest or the fastest unless the comparison supports

that claim. A separate audit, described in Chapter 5, checks the correspondence between explanations and the logged values.

4.14 Logging, Configuration and Reproducibility

Every experiment is described by a configuration file that lists the scenario parameters, the reward weights, the algorithm and its hyperparameters, the random seeds, and the code version. The configuration is stored with the output so that the experiment can be repeated. Logs contain the state summary, feasible candidates, selected action, reward components, outcome, and explanation for each decision.

Paired comparisons require that all methods experience the same sequence of orders and the same traffic realisation. This is achieved by generating the scenario once from the seed and replaying it for each method. Differences between methods then reflect their decisions and not chance variation in the input.

4.15 Implementation Plan

The implementation is planned in phases so that each component can be verified before the next is added.

- Phase 1: environment skeleton with order, partner, and traffic entities, together with the scenario generator and unit tests for the entities.
- Phase 2: distance and travel-time model, risk module, and reward function, tested with hand-constructed examples.
- Phase 3: baselines, including the nearest-partner rule and the Hungarian Algorithm, used to validate the environment before learning begins.
- Phase 4: DRL agent training with action masking, validation, and checkpointing.
- Phase 5: explanation module, evaluation scripts, and report generation.
- Phase 6: sensitivity analysis, stress tests, and documentation of limitations.

Completing the baselines before training the agent is deliberate. If the baselines behave sensibly, the environment is more likely to be correct, and any later surprise in the learned policy can be attributed with more confidence to learning rather than to a simulator error.

4.16 Worked Example of One Assignment Step

A small numerical example illustrates how the components of the framework interact. Suppose a new order arrives with a moderate deadline and three partners are eligible. Partner A is closest but already carries two active orders. Partner B is slightly farther and carries one order. Partner C is the farthest and has no active order. The state builder scales the distances, estimated travel times, and workloads and passes them to the policy together with the feasibility mask.

The risk module computes a score for each pair. Because partner A has the highest workload and its route crosses a congested zone, its risk score is highest. Partner C has the lowest risk but the longest travel time. Partner B lies between the two on both measures. The nearest-partner rule would select A because of distance alone. The Hungarian Algorithm would select the pair with the lowest combined cost for the supplied matrix, which may be B or C depending on the weights.

A trained policy would select the action with the highest learned value, which also reflects expected future orders. If the explanation layer finds that the selected partner has a lower travel time than C and a lower risk than A, it produces a sentence of this kind and records the three compared values. The example is illustrative only; it does not represent a measured result, and the numbers in a real run depend on the scenario, the weights, and the trained policy.

4.17 Pseudocode of the Decision Loop

The overall decision loop of the framework can be summarised in the following steps, which are the same for the learned policy and for the baselines apart from the way the action is chosen. • Initialise the environment from the configuration and seed, and obtain the first observation.

- Build the state and the feasibility mask from the pending orders, partner status, and traffic profile.
- Compute travel-time and risk estimates for each feasible pair.
- Ask the selected method for an action; if no feasible action exists, record a deferral.
- Generate the explanation from the compared feature values.
- Apply the action, advance the simulation clock, update partner workload and order status, and introduce new arrivals.
- Compute the reward components and write the decision record.
- Repeat until the end of the simulated period, then aggregate the metrics.

Writing the loop in this explicit form makes it clear that the learned policy and the baselines differ in only one step. This property supports the fairness of the comparison and simplifies the testing of the common parts of the system.

CHAPTER 5
EVALUATION, TESTING AND DISCUSSION

5.1 Evaluation Metrics

Evaluation is planned across the same simulated scenarios for all methods. The metrics represent service quality, travel efficiency, utilisation, risk, and transparency. No measured values are reported because the source paper identifies the project as being at the design stage. Criterion Measure Purpose Delivery time Mean elapsed time per completed order Service speed On-time rate Orders completed by deadline / completed orders Deadline reliability Assignment agreement Agreement with Hungarian assignments on static snapshots Similarity to classical baseline Total distance Sum of partner travel distance Travel efficiency Workload distribution Spread or variance of work across partners Utilisation balance Risk score Mean normalised risk score of selected assignments Modelled operational risk Explanation coverage Assignments with explanation / total assignments Transparency Feasibility rate Valid assignments / attempted assignments Constraint handling

5.2 Experimental Design

Experiments should cover light, moderate, and heavy order loads, as well as normal and congested traffic. Additional scenarios can vary partner availability, order location spread, deadline tightness, and the proportion of high-priority orders. The same scenario definitions and, where possible, random seeds should be used for every method. The DRL policy should be frozen during final evaluation, and the test set should not be used for tuning.

Results should include the number of trials, mean, variability or confidence interval, and any failed or infeasible runs. If the DRL method improves one metric but worsens another, the trade- off must be stated. A claim of risk reduction requires measured risk outcomes under equivalent conditions.

5.3 Test Plan

The following tests are planned validation activities. Their status must be updated only after the implementation has actually been run. ID Test Expected outcome Status T-01 Order generation Valid locations, deadlines, priority and arrival times Planned T-02 Partner availability Unavailable partners excluded from feasible actions Planned T-03 Risk normalisation Each risk component follows documented scale Planned T-04 No feasible partner Defined no-assignment or deferral outcome Planned T-05 Deadline pressure Urgency affects delay-related reward as configured Planned T-06 Traffic changes Travel time responds to traffic multipliers Planned T-07 Baseline parity Methods receive equivalent scenario information Planned T-08 Assignment validity Selected partner is eligible and available Planned T-09 Explanation generation Each assignment receives a grounded explanation Planned T-10 Reproducibility Same seed/configuration reproduces scenario inputs Planned T-11 Held-out evaluation Test scenarios excluded from policy tuning Planned T-12 Metric aggregation Metrics and run identifiers retained consistently Planned

5.4 Expected Analysis and Limitations

The framework may offer a flexible way to study changing delivery conditions because the policy can learn from sequential feedback. The risk module makes operational risk explicit, workload balancing aims to reduce task concentration, and the explanation layer may improve auditability. These are design expectations, not demonstrated findings. A simulator approximates real traffic, order patterns, and partner behaviour; strong simulated performance may not transfer directly to a city deployment. Reward weights influence learned behaviour, DRL does not guarantee global optimality, and risk scores depend on input quality and assumptions. Explanations may be misleading if they do not faithfully reflect the decision process. Future evaluation should include sensitivity analysis, stress tests, explanation audits, and eventually a limited pilot using real order and traffic data with appropriate privacy controls and a fallback dispatch procedure.

5.5 Scenario Matrix

The experimental scenarios are obtained by combining the factors introduced in Section 5.2. The factors and their proposed levels are listed below. The complete matrix may be reduced to a fractional design if computing time is limited, but the reduction must be documented.

- Order load: light, moderate, and heavy.
- Traffic: normal and congested.
- Deadline tightness: relaxed and tight.
- Spatial layout: uniform and clustered.
- Priority share: low and high.

Each combination is repeated for several random seeds. Reporting results by scenario avoids the situation in which a method that excels in easy cases hides poor performance in demanding ones. Particular attention is given to the heavy, congested, tight-deadline combination, because this is where differences between methods are expected to become visible.

5.6 Statistical Treatment of Results

For each metric the report should present the mean over seeds together with a measure of spread, such as the standard deviation or a confidence interval. Because all methods are run on the same scenarios, paired comparisons are appropriate: the difference between two methods is computed for each seed and summarised, which reduces the effect of scenario difficulty.

Where the number of seeds is small, non-parametric procedures or bootstrap intervals are preferable to assumptions of normality. A difference should be described as meaningful only if it is larger than the variability between seeds. Failed, infeasible, or truncated runs must be counted and reported rather than silently removed.

5.7 Planned Ablation Studies

Ablation studies remove or modify one component at a time to estimate its contribution. The following variants are proposed.

- Without the risk term: the reward and state omit the risk score, showing whether explicit risk affects delay and workload.
- Without workload balancing: the imbalance penalty is set to zero, showing its influence on utilisation.
- With reduced state: traffic or deadline features are removed to measure their value to the agent.
- With different algorithms: DQN is compared with PPO under the same budget.

Because a learned policy may adapt to remove a penalty in unintended ways, ablation results should be read together with the individual metric values and not only with the cumulative reward.

5.8 Evaluation of Explanations

Explanation quality is evaluated separately from decision quality. Coverage measures the proportion of assignments that have an explanation. Fidelity measures whether the factors mentioned in the explanation correspond to the logged values; this can be checked automatically by recomputing the comparison. Consistency measures whether similar situations receive similar explanations.

Human review can complement the automatic checks. A small group of reviewers may read a sample of explanations and judge whether they are clear and whether they would help in accepting or rejecting the recommendation. Because such a study involves people, it should be described as a future activity and conducted with appropriate consent.

5.9 Threats to Validity

Internal validity may be reduced by implementation errors, unequal treatment of methods, or tuning on the test scenarios. These risks are mitigated by shared interfaces, paired scenarios, separation of training, validation, and test seeds, and early validation of the environment with simple baselines.

External validity concerns how far simulation results apply to real operations. The synthetic traffic and order models may not capture local road networks, driver behaviour, or demand patterns. Statements about real-world benefit are therefore outside the scope of the present study and would require validation against historical data and a controlled pilot.

Construct validity concerns whether the metrics measure what matters. A low mean delivery time may hide a few very late orders, and a high on-time rate may be achieved by rejecting difficult orders. Reporting several complementary metrics, including percentiles of delivery time and the number of deferrals, reduces this risk.

5.10 Acceptance Criteria for the Prototype

Before the framework is described as complete, the following criteria should be satisfied. The simulator reproduces the same scenario from the same seed. All planned tests in Section 5.3 have been run and their status updated. Each method produces only feasible assignments or defined deferrals. Every assignment has an explanation that passes the fidelity check. Results are reported for all scenario groups with the number of seeds and a measure of variability. Any limitation that affects interpretation is stated in the report.

The criteria deliberately do not require the learned policy to outperform the baselines. A well-documented outcome in which the learned policy performs better on some metrics and worse on others is a valid and useful result.

5.11 Reporting Format for Results

To keep the final results readable and comparable, a fixed reporting format is proposed. For each scenario group the report should contain a summary table with one row per method and one column per metric, giving the mean and the measure of spread over seeds. A second table should show paired differences between the learned policy and each baseline. Figures may show the distribution of delivery times, the distribution of orders across partners, and the evolution of the reward components during training.

The text accompanying each table should describe what the numbers show without overstating them. Where the learned policy is better on one metric and worse on another, both facts must appear in the same paragraph. Where a result is within the variability between seeds, it should be described as inconclusive and not as an improvement.

5.12 Resources and Runtime Considerations

The prototype is designed to run on a standard development computer. Training time depends on the network size, the number of environment steps, and the cost of each simulator step. Because the simulator is implemented in software and does not call external services, each step is inexpensive, and many episodes can be run in parallel for different seeds. The evaluation phase is also lightweight, since the policy is frozen and only needs to be applied.

Runtime of the decision methods should be measured as well as quality. The time required to select an assignment is important in operations, where a recommendation must be returned quickly. The Hungarian Algorithm, the nearest-partner rule, and the neural network policy are expected to differ in decision latency, and this difference should be recorded under comparable hardware conditions.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion

SmartRoute AI is proposed as an explainable and risk-aware DRL framework for dynamic last-mile delivery assignment. It combines order requests, partner status, traffic estimates, distance, deadlines, and priority in a state representation. A DRL agent selects feasible assignments and receives reward feedback based on delay, distance, operational risk, and workload imbalance, with a bonus for on-time delivery. The Hungarian Algorithm provides a classical baseline and a nearest-partner rule offers a simple comparison point.

The framework includes an explanation layer that reports why a partner was selected using travel-time estimates, risk, and workload. Its modular architecture is intended for evaluation in a software simulator using repeatable scenarios and common metrics. Its contribution is the integration of dynamic learning, explicit risk awareness, workload balance, baseline comparison, and per-decision explanation in one proposed system. No numerical improvement is claimed because the supplied source material does not include completed simulation results.

6.2 Future Enhancements

Future work should complete the simulator, train candidate DQN and PPO agents, run controlled experiments across order intensity and traffic, perform sensitivity analysis on reward weights, and validate the simulator against historical delivery data. Further extensions may jointly optimise assignment and route planning, explore multi-agent coordination, integrate a dispatcher dashboard, audit explanation quality, test robustness to missing data and abrupt traffic changes, and conduct a limited real-world pilot with operational safeguards.

6.3 Recommendations for Final Validation

Before deployment, acceptance thresholds should be agreed for service quality, feasibility, explanation quality, runtime performance, and risk handling. A fallback method should be available when no feasible assignment exists or the learned policy is unsuitable. Model updates should be versioned and tested against a fixed benchmark suite. Any operational claim should be supported by measurements from the relevant environment.

1. Confirm every order has a unique identifier and valid deadline.
2. Reject partners that are unavailable or have no remaining capacity.
3. Check that distance and ETA are finite, non-negative values.
4. Validate traffic and risk feature ranges before computing costs.
5. Ensure each assignment corresponds to a feasible candidate.
6. Commit order and partner updates in one database transaction.
7. Store decision records with model and configuration versions.
8. Test failed logins, session expiry, invalid tokens and rate limiting.
9. Compare all algorithms on the same held-out scenarios.
10. Report means and confidence intervals, not one favourable run.
11. Separate synthetic simulation results from real-world claims.
12. Document limitations: approximate road distance, synthetic traffic, design-selected weights, one simulation world per server process, and no implemented GAT or SHAP functionality in the current prototype.

6.4 Ethical and Social Considerations

Automated assignment affects people. Partners depend on assignments for income, and a policy that systematically sends difficult or poorly paid orders to the same individuals would be unfair. The workload term in the reward is intended to reduce this concentration, but fairness should also be measured directly, for example by comparing the distribution of orders and distances across partners.

Transparency toward partners is also important. They should be able to understand the general rules used to allocate work and to raise concerns about particular decisions. The decision records and explanations proposed in this report can support such a process, provided that they are presented in an understandable form.

Finally, the system should be used as decision support. A human dispatcher should be able to override a recommendation, and a fallback rule should be available when the learned policy is unsuitable. These safeguards reduce the harm that could result from an unforeseen failure of the model.

6.5 Summary of Contributions

This report has proposed an integrated framework for dynamic last-mile delivery assignment. Its contributions are the formulation of assignment as a sequential decision problem, a risk-aware reward with separately logged components, a feasibility-masked action space, a fair baseline comparison with the Hungarian Algorithm and a nearest-partner rule, a template-based explanation procedure grounded in measured feature differences, and an evaluation plan that emphasises reproducibility and honest reporting.

The next step is to implement the simulator and agent as outlined, execute the planned tests, and report the measured outcomes with their limitations. Only after those results are available can the benefits of the framework be assessed.

6.6 Lessons for Practice

Several practical lessons follow from the design process. First, the simulator is as important as the learning algorithm; an inaccurate environment can mislead the agent and the reader. Second, simple baselines are valuable because they expose errors in the environment and provide a reference for judging complexity. Third, a multi-term reward must be monitored component by component, since the sum can conceal undesirable behaviour. Fourth, explanations should be generated from the same data as the decision and checked automatically.

Finally, honest reporting is part of engineering quality. A framework that is described accurately, with its assumptions and limitations, is more useful to a reader than one that claims unsupported gains. The structure of this report was chosen to keep that distinction visible.

A.1 Geographic Distance and Road-Factor Approximation

These excerpts illustrate the corelogic described inthe project report and README. They are representative, self-contained teaching excerpts rather than verbatim copies of the unavailable source files. The documented project uses a FastAPI backend, NumPy-based PPO implementation, synthetic traffic simulation, risk-aware assignment and an explainability layer.

REFERENCES

[1] N. Boysen, S. Fedtke, and S. Schwerdfeger, “Last-mile delivery concepts: A survey from an operational research perspective,” OR Spectrum, vol. 43, 2021.

[2] H. W. Kuhn, “The Hungarian method for the assignment problem,” Naval Research Logistics Quarterly, vol. 2, no. 1–2, pp. 83–97, 1955.

[3] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.

[5] M. Nazari, A. Oroojlooy, L. Snyder, and M. TakáI, “Reinforcement learning for solving the vehicle routing problem,” in Advances in Neural Information Processing Systems, 2018.

[6] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!,” in Proceedings of ICLR, 2019.

[7] Z. Xu et al., “Large-scale order dispatch in on-demand ride-hailing platforms: A learning and planning approach,” in Proceedings of ACM SIGKDD, 2018.

[8] K. Lin, R. Zhao, Z. Xu, and J. Zhou, “Efficient large-scale fleet management via multi-agent deep reinforcement learning,” in Proceedings of ACM SIGKDD, 2018.

[9] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems, 2017.

[10] M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in Proceedings of ACM SIGKDD, 2016.

[11] E. Puiutta and E. M. S. P. Veith, “Explainable reinforcement learning: A survey,” in Proceedings of CD-MAKE, 2020.

[12] A. Heuillet, F. Couthouis, and N. Díaz-Rodríguez, “Explainability in deep reinforcement learning,” Knowledge-Based Systems, vol. 214, 2021.

[13] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” Journal of Machine Learning Research, vol. 16, pp. 1437–1480, 2015.

[14] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in Proceedings of ICML, 2017.

[15] G. B. Dantzig and J. H. Ramser, “The truck dispatching problem,” Management Science, vol. 6, no. 1, pp. 80–91, 1959.

[16] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[17] G. Brockman et al., “OpenAI Gym,” arXiv:1606.01540, 2016.

[18] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems, 2019.

[19] A. Raffin et al., “Stable-Baselines3: Reliable reinforcement learning implementations,” Journal of Machine Learning Research, vol. 22, 2021.